

RUTILLAS — PRODUCTION TECH SPEC (V2 - SQUARE)

1. Architecture

Frontend: React + Vite

Backend: Django + Django REST Framework

Database: PostgreSQL (Heroku Postgres)

Auth: Django Auth + JWT

Payments: Square (Checkout + Webhooks)

2. Database Models

```
class User(AbstractUser):
    region = models.CharField(max_length=100)
    latitude = models.FloatField()
    longitude = models.FloatField()
    is_subscribed = models.BooleanField(default=False)
    is_free_user = models.BooleanField(default=False)

class Subscription(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    status = models.CharField(max_length=20)
    start_date = models.DateTimeField()
    end_date = models.DateTimeField()
    square_payment_id = models.CharField(max_length=255)

class Route(models.Model):
    title = models.CharField(max_length=255)
    description = models.TextField()
    city = models.CharField(max_length=100)
    date = models.DateTimeField()
    max_participants = models.IntegerField()
    creator = models.ForeignKey(User, on_delete=models.CASCADE)
    status = models.CharField(max_length=20, default="active")

class RouteParticipant(models.Model):
    route = models.ForeignKey(Route, on_delete=models.CASCADE)
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    joined_at = models.DateTimeField(auto_now_add=True)

class Follow(models.Model):
    follower = models.ForeignKey(User, related_name='following', on_delete=models.CASCADE)
    following = models.ForeignKey(User, related_name='followers', on_delete=models.CASCADE)
```

3. API Endpoints

```
POST /auth/register
POST /auth/login
GET /routes
POST /routes
POST /routes/{id}/join
GET /routes?city=Barcelona
POST /follow/{user_id}
POST /unfollow/{user_id}
POST /payments/create-link
POST /payments/webhook
```

4. Square Payment Flow

1. Frontend requests payment link from backend
2. Backend generates Square checkout link
3. User completes payment
4. Square sends webhook to backend
5. Backend activates subscription

```
if payment.status == "COMPLETED":  
    user.is_subscribed = True  
    subscription.end_date = now + 30 days
```

5. Subscription Enforcement

```
if not user.is_subscribed and not user.is_free_user:  
    return error
```

6. Free Users Logic

```
if distance <= 30km and free_users_count < 100:  
    user.is_free_user = True
```

7. Join Logic

```
if route.participants.count() >= route.max_participants:  
    return error  
  
create participant  
  
if count == max:  
    route.status = "full"
```

8. Notifications

```
Notification.objects.create(  
    user=route.creator,  
    message="New participant joined"  
)
```

9. Cleanup

Delete routes 48h after event ends using cron job.

10. Admin

Admin uses Django Admin panel to manage users, subscriptions, and revenue.